



Airline



STACK TECNOLÓGICO DE MICROSERVICIOS



Taller AeroReserva como ejemplo práctico

Grupo 5

SARA MUÑOZ
SAMUEL DAZA
SAMUEL VELANDIA





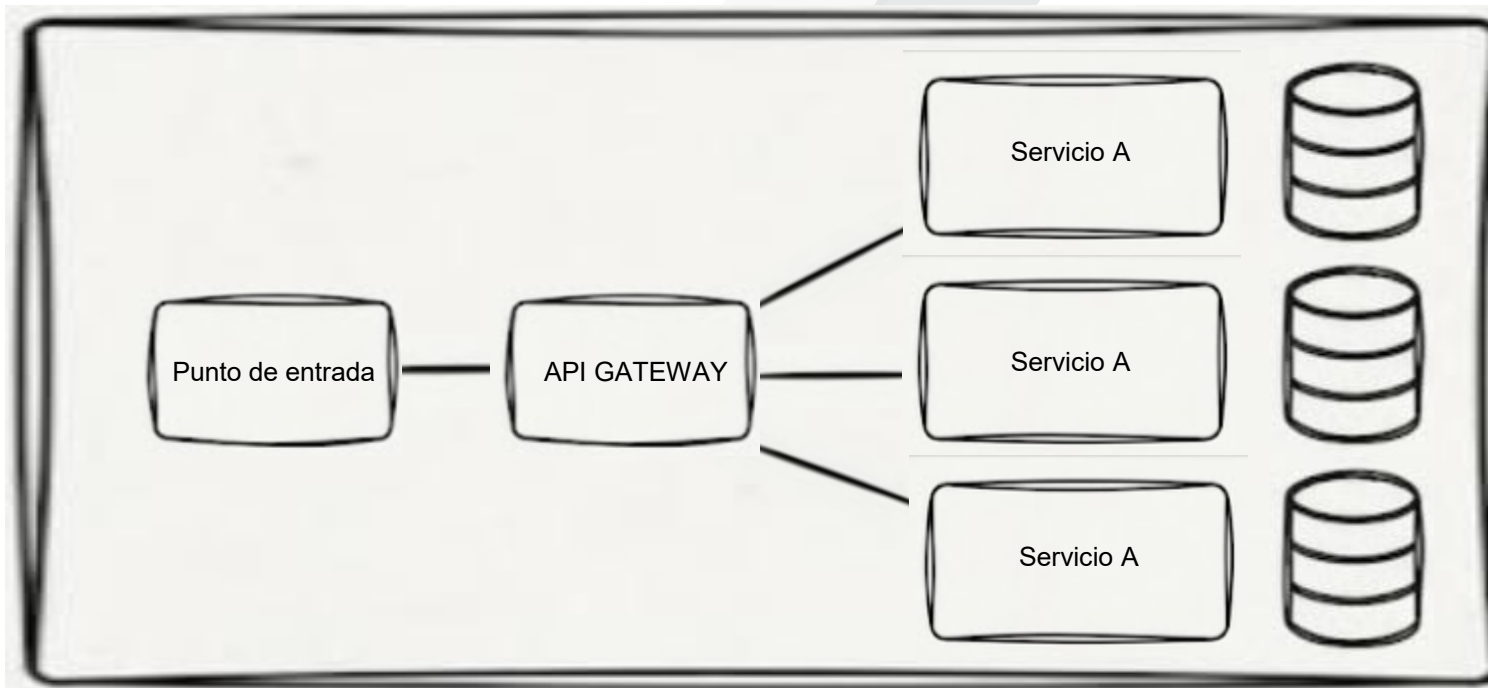
Airline



INVESTIGACIÓN



MICROSERVICIOS

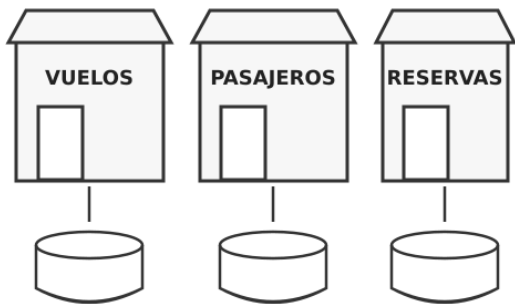


Alternativa: monolito modular. Menos operación distribuida, pero no cumple el estilo requerido.

MICROSERVICIOS: COMPARACIÓN

MICROSERVICIOS

Locales con responsabilidades propias

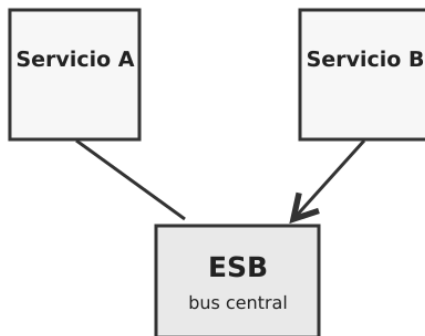


API definidas entre locales

Cada uno guarda sus datos y puede desplegarse por separado.

SOA TRADICIONAL

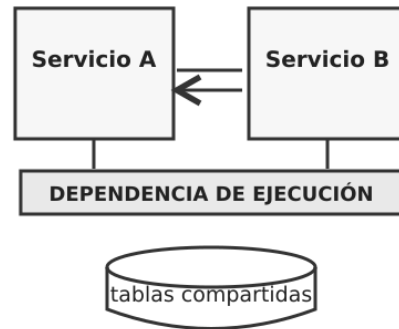
Oficinas con central de comunicaciones



La central concentra enrutamiento y transformación; a veces, lógica.

MONOLITO DISTRIBUIDO

Locales separados que operan atados



Cambiar o desplegar uno exige coordinar también al otro.

MICROSERVICIOS: COSTOS

DIMENSIÓN	QUÉ CAMBIA	CONSECUENCIA
Rendimiento	Una llamada cruza la red.	Más latencia y serialización.
Datos	No hay una transacción local única.	Coordinar estados y compensaciones.
Operación	Hay varios procesos y despliegues.	Monitoreo, secretos y automatización.
Equipos	Los límites cruzan equipos.	Más coordinación si nadie es dueño.
Pruebas	El resultado depende de la red.	Integrar y simular fallos cuesta más.
Resiliencia	Un servicio puede degradarse.	Timeout, retry y circuit breaker.
Proliferación	Crecen servicios y dependencias.	Rastrear el flujo se vuelve difícil.

MICROSERVICIOS: PATRONES

PATRONES FRECUENTES

API Gateway / BFF

Un punto recibe solicitudes de clientes y las adapta.
Cuando hay clientes externos

Datos por servicio

Cada servicio controla sus datos sin acceso compartido.
Autonomía de los datos

Saga

Coordina pasos y compensa si una operación falla.
Proceso entre servicios

Circuit Breaker

Corta llamadas repetidas hacia un servicio degradado.
Dependencia síncrona

CUANDO CAMBIA EL CONTEXTO

CQRS → lecturas y escrituras necesitan modelos diferentes.

Event Sourcing → el historial de eventos es la fuente de verdad.

Bulkhead → aislar recursos de una dependencia que puede agotarlos.

Strangler Fig → reemplazar gradualmente un monolito existente.

DOMA → ordenar muchos servicios por dominios y gateways.

DOS CASOS REALES

Netflix · fallo de base de datos en 2008 → migración gradual a AWS.

Uber · ~2.200 microservicios críticos en 2020 → DOMA para ordenar dependencias.

ASTRO PARA LA INTERFAZ



Mar de HTML: contenido visible desde el inicio

Barco A: JS ahora



Isla A

Encabezado interactivo

Funciona por separado

Isla B

Carrusel de imágenes

No bloquea el encabezado



Barco B: JS al verlo
opción client:visible

Cada isla recibe solo el JavaScript que necesita.

Menos JavaScript

Islas independientes

Carga cuando conviene

ASTRO: VENTAJAS Y DESVENTAJAS



Conviene usarlo

Ventajas

- Controla cuánto JavaScript llega al navegador.
- Reúne páginas, componentes y rutas de servidor.

Ejemplos adecuados

Documentación, blogs, catálogos y formularios con interacciones puntuales.

Conviene comparar alternativas

Límites

- Aporta menos si casi toda la pantalla cambia y comparte estado continuamente.
- Las páginas dinámicas necesitan un servidor.

Ejemplo para evaluar

Una aplicación con interacción constante en prácticamente toda la interfaz.

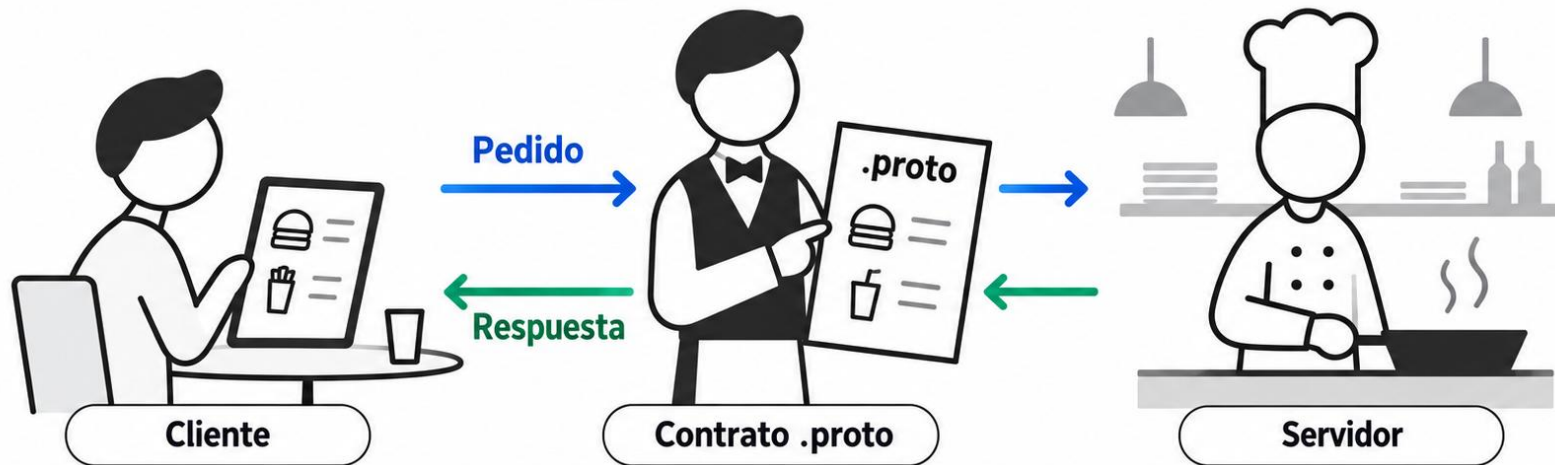
Microsoft · Fluent 2

Sitio del sistema de diseño y sus ejemplos.

Firebase · Blog

Publicación de contenido más ágil.

gRPC Y PROTOCOL BUFFERS



Mismo idioma



Rápido



Ordenado

gRPC: VENTAJAS Y DESVENTAJAS



Conviene usarlo

Ventajas

- Contratos estrictos con archivos .proto.
- Comunicación rápida con HTTP/2 + Protobuf.
- Soporta Unary y Streaming.

Casos ideales

Comunicación backend entre microservicios, streaming en tiempo real, IoT y sistemas distribuidos.

Conviene comparar alternativas

Límites

- Depuración más difícil por tráfico binario.
- En navegadores suele requerir gRPC-Web y proxy.
- Curva de aprendizaje y gobernanza estricta de esquemas.
- Puede ser excesivo para CRUD simples o APIs públicas sencillas.

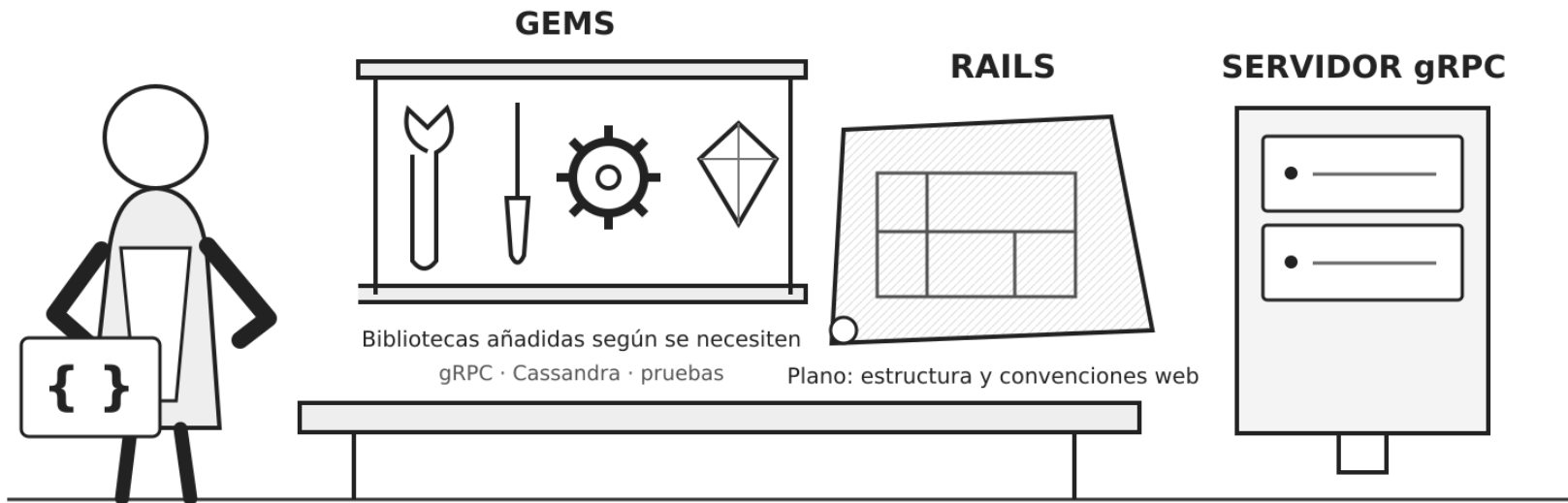
Ejemplo para evaluar

Frontends web con mucha interacción suelen preferir REST/JSON o GraphQL en vez de gRPC directo.

Casos en la industria

Google, Netflix y Square lo usan para comunicación interna de alto rendimiento entre servicios.

RUBY Y RAILS



RUBY

Lenguaje: clases y módulos
construyen la lógica.

Proceso Ruby independiente
guiado por el contrato .proto

RUBY Y RAILS: VENTAJAS Y DESVENTAJAS •



A favor

Ruby: lógica legible + gems para integrar.

Rails: estructura y convenciones web.

Útil cuando esas herramientas ahorran trabajo.

A evaluar

Rails puede sobrar en un proceso pequeño.

Rendimiento: medir con la carga esperada.

El lenguaje por sí solo no predice el resultado.

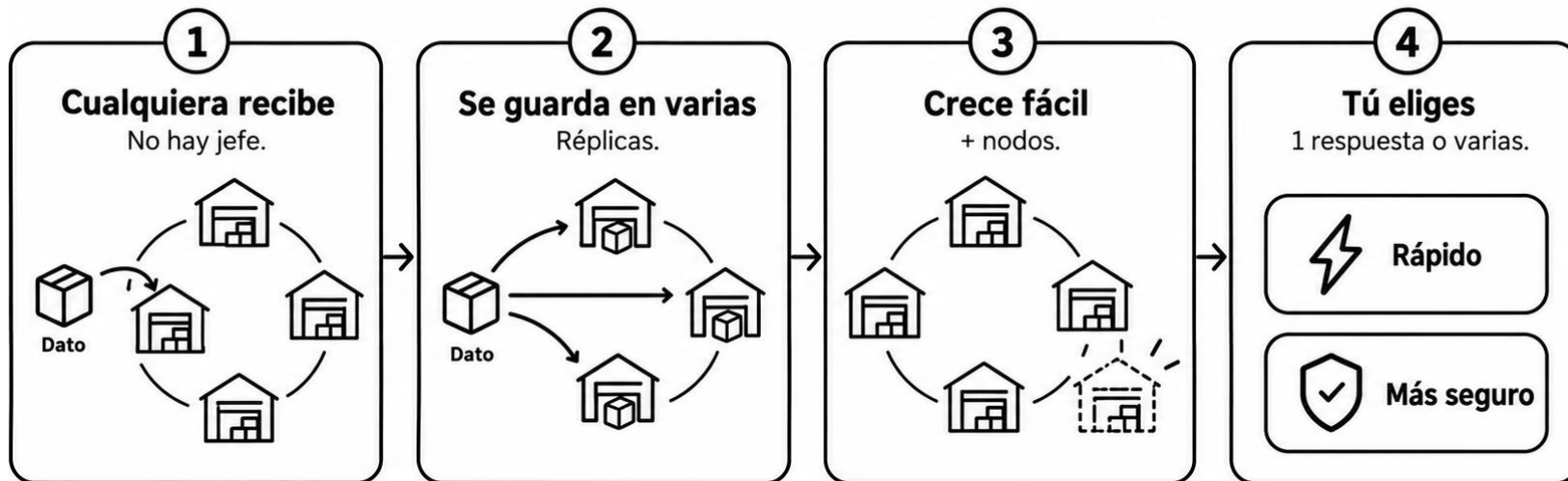
Shopify • Plataforma principal

Ruby y Rails en partes centrales de la plataforma.

Shopify • Aplicación Shop

Backend desarrollado con Ruby on Rails.

CASSANDRA Y DATOS



Distribuida

•



Persistente

•



Escalable

KEYSPACES DE CASSANDRA



aeroreserva_flights

Tablas de vuelos

aeroreserva_passengers

Tablas de pasajeros

aeroreserva_bookings

Tablas de reservas

Una instancia de Cassandra puede tener varios keyspaces.

Cada servicio usa su propio espacio lógico.



Organización

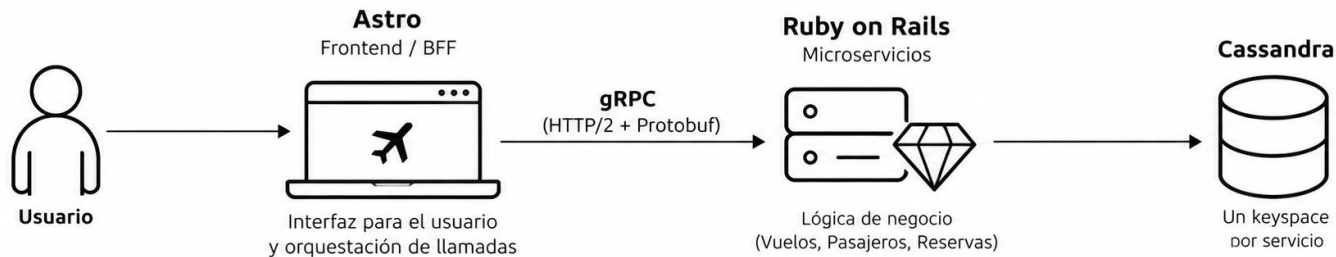


Aislamiento



Replicación

STACK JUNTO



Astro

Frontend / BFF



- + Interfaz eficiente y BFF ligero
- Menos componentes que otros frameworks
- Se elige por su simplicidad y eficiencia para el frontend del proyecto

Ruby on Rails

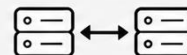
Microservicios backend



- + Desarrollo rápido y estructura organizada
- Mayor configuración que opciones más ligeras
- Se elige por sus convenciones y facilidad de desarrollo

gRPC

Comunicación entre servicios



- + Comunicación eficiente y contratos tipados
- Mayor complejidad y soporte limitado en navegador
- Se elige para la comunicación interna entre microservicios



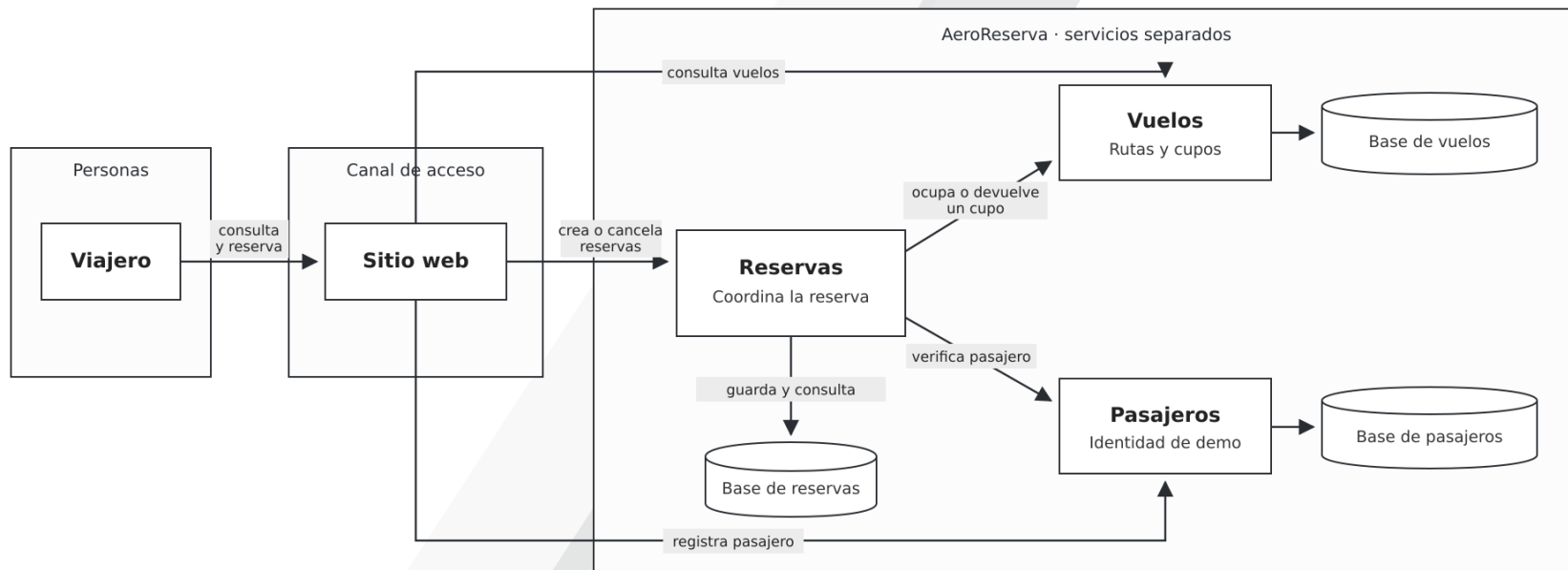
Airline



AERORESERVA



ARQUITECTURA DE ALTO NIVEL

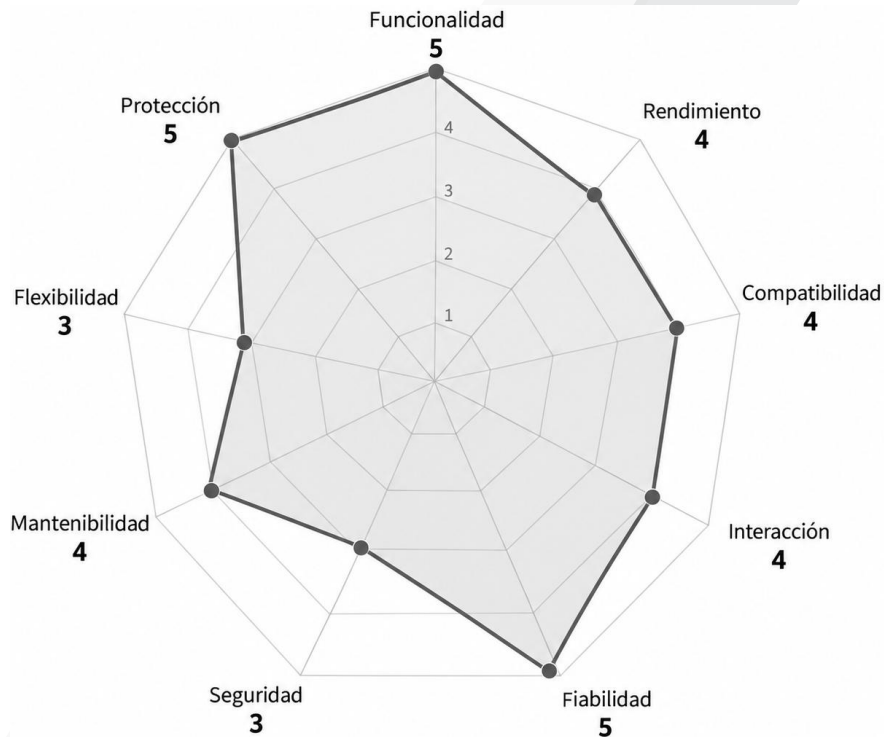


REQUISITOS FUNCIONALES



ID	Requisito
RF-01	Consultar el catálogo de vuelos, sus rutas, horarios y cupos disponibles.
RF-02	Filtrar vuelos por origen y destino.
RF-03	Registrar e identificar a un pasajero para la demostración.
RF-04	Crear una reserva para el pasajero identificado y un vuelo con cupos.
RF-05	Consultar únicamente las reservas asociadas al pasajero identificado.
RF-06	Cancelar una reserva propia y restituir el cupo correspondiente.
RF-07	Informar al usuario si una operación falla, sin presentar identificadores técnicos como pasos obligatorios del flujo.

ATRIBUTOS DE CALIDAD





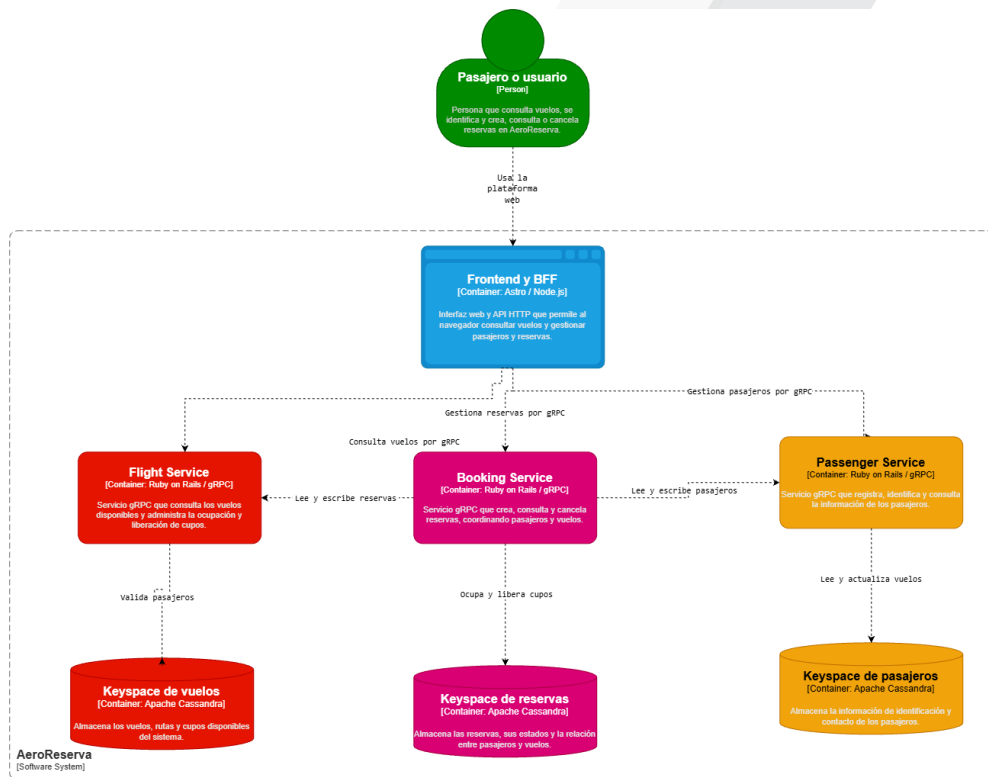
Airline



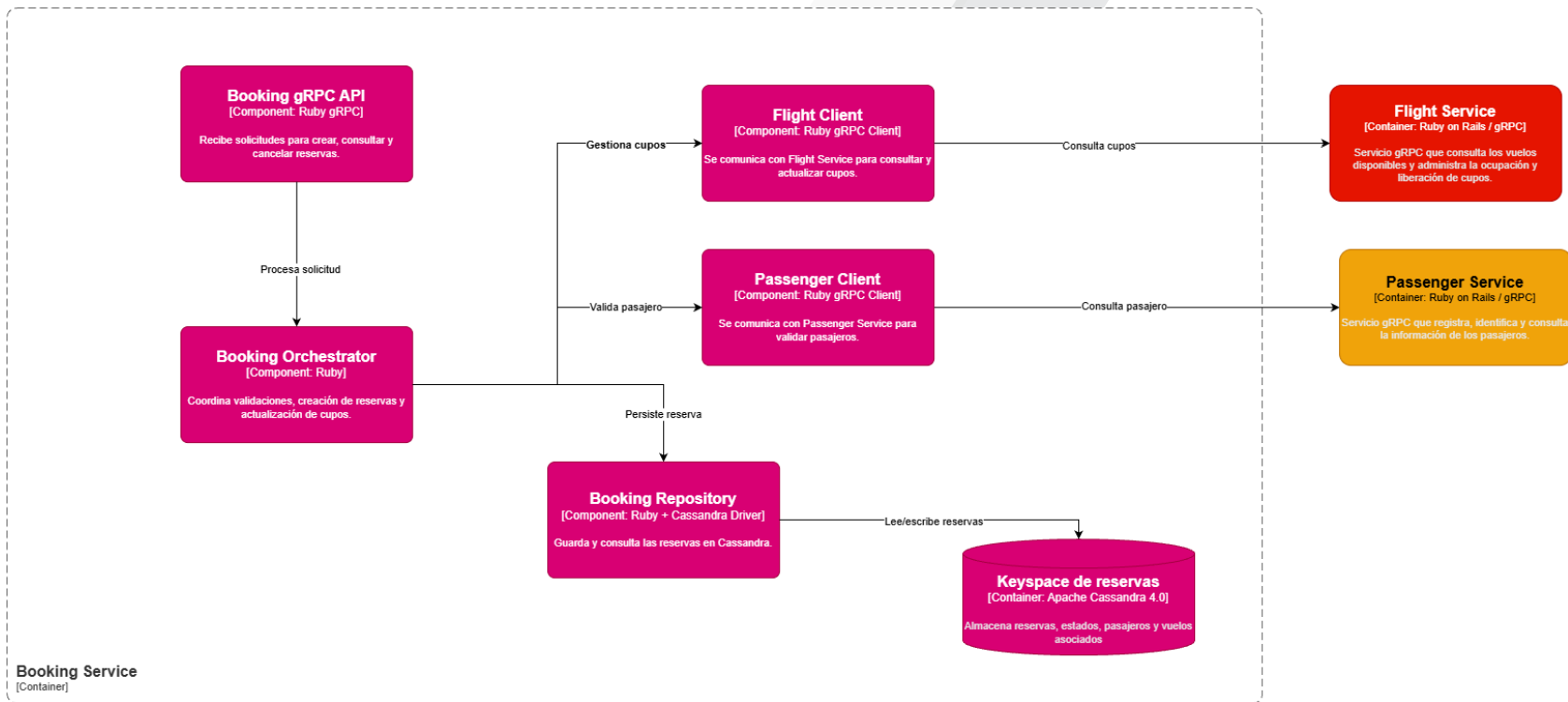
C4 CONTEXTO



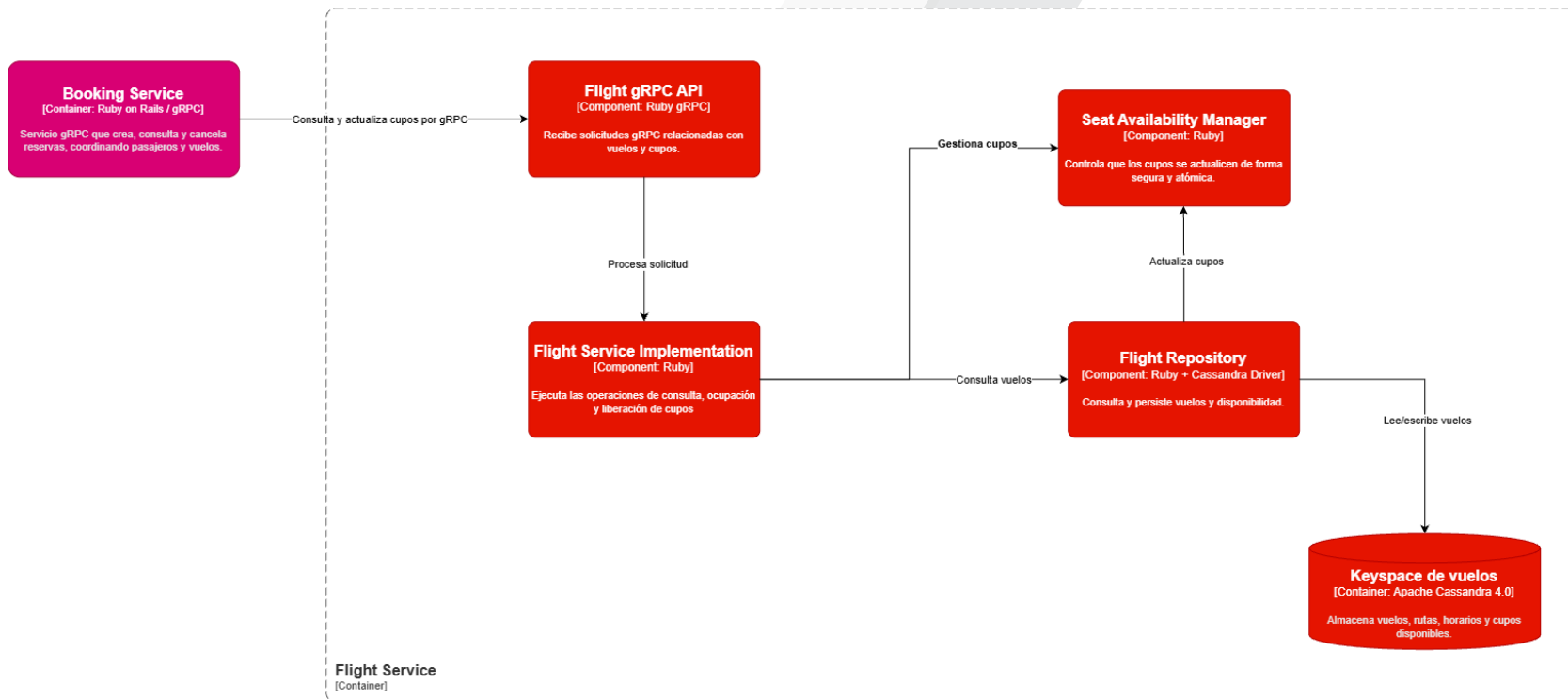
C4 CONTENEDORES



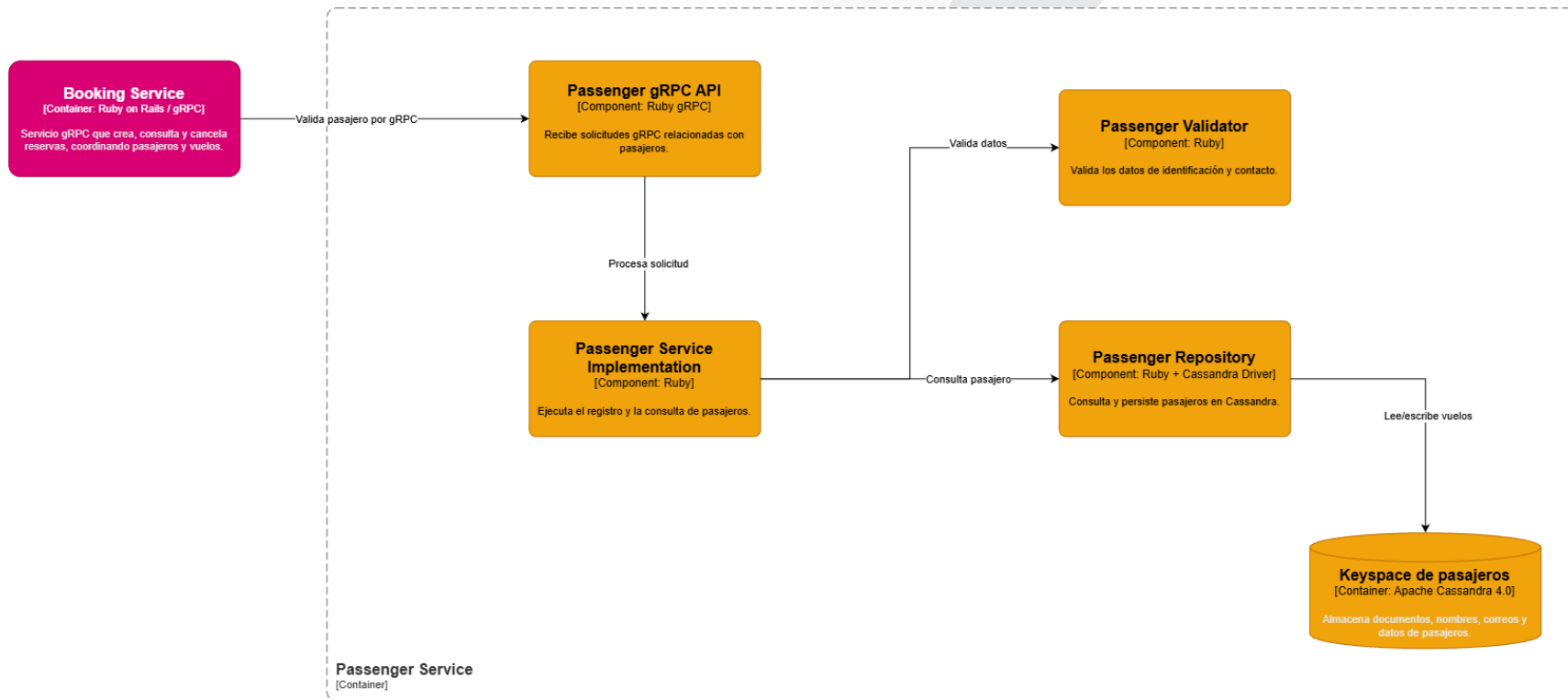
C4 BOOKING SERVICE



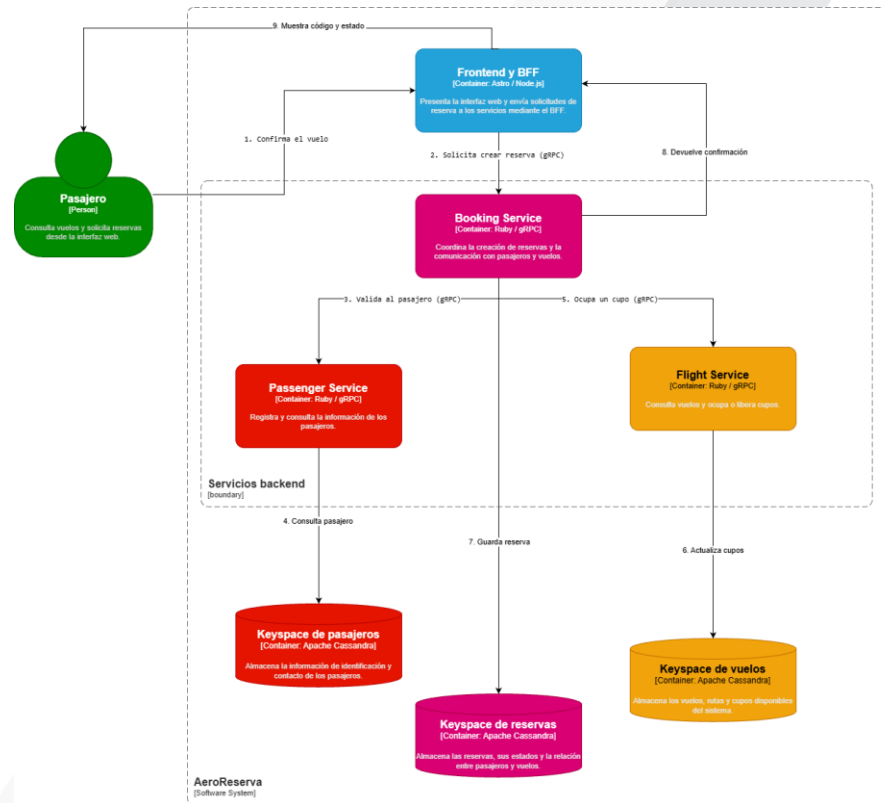
C4 FLIGHT SERVICE



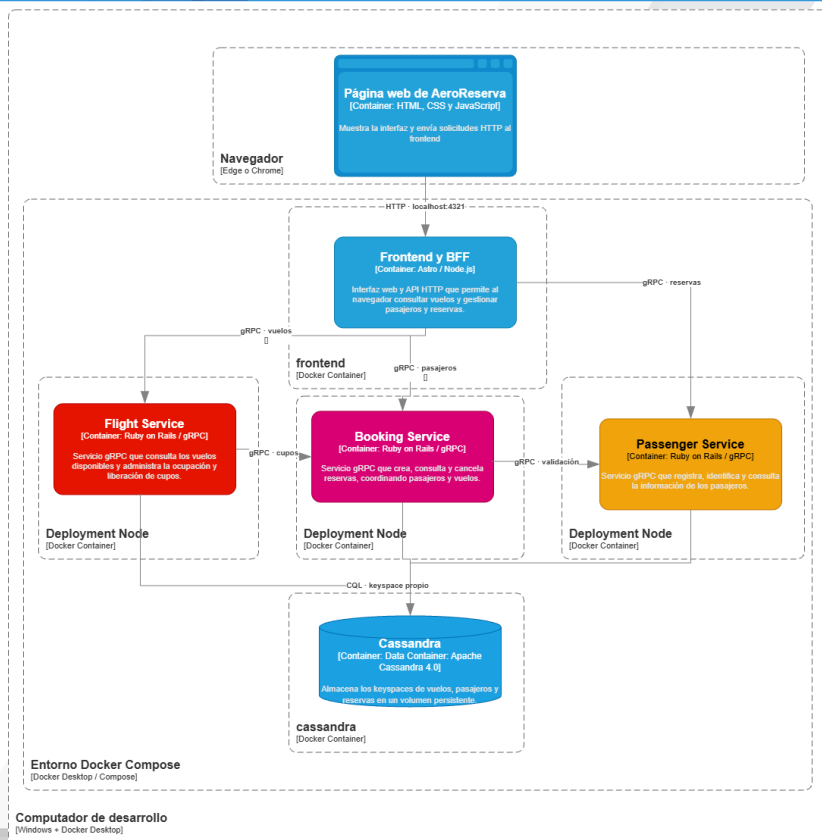
C4 PASSENGER SERVICE



C4 RESERVA EN TIEMPO DE EJECUCIÓN



C4 DESPLIEGUE LOCAL





Airline



MODELO DE DATOS

Vuelos

flights_by_id
flights_catalog

Pasajeros

por ID
por documento

Reservas

por ID
por pasajero
por estado



Airline



PATRONES Y RIESGOS

Aplicados

BFF

Datos por servicio

Contrato primero

Saga

Riesgos controlados

Llamadas síncronas

Cassandra física compartida

Observabilidad limitada



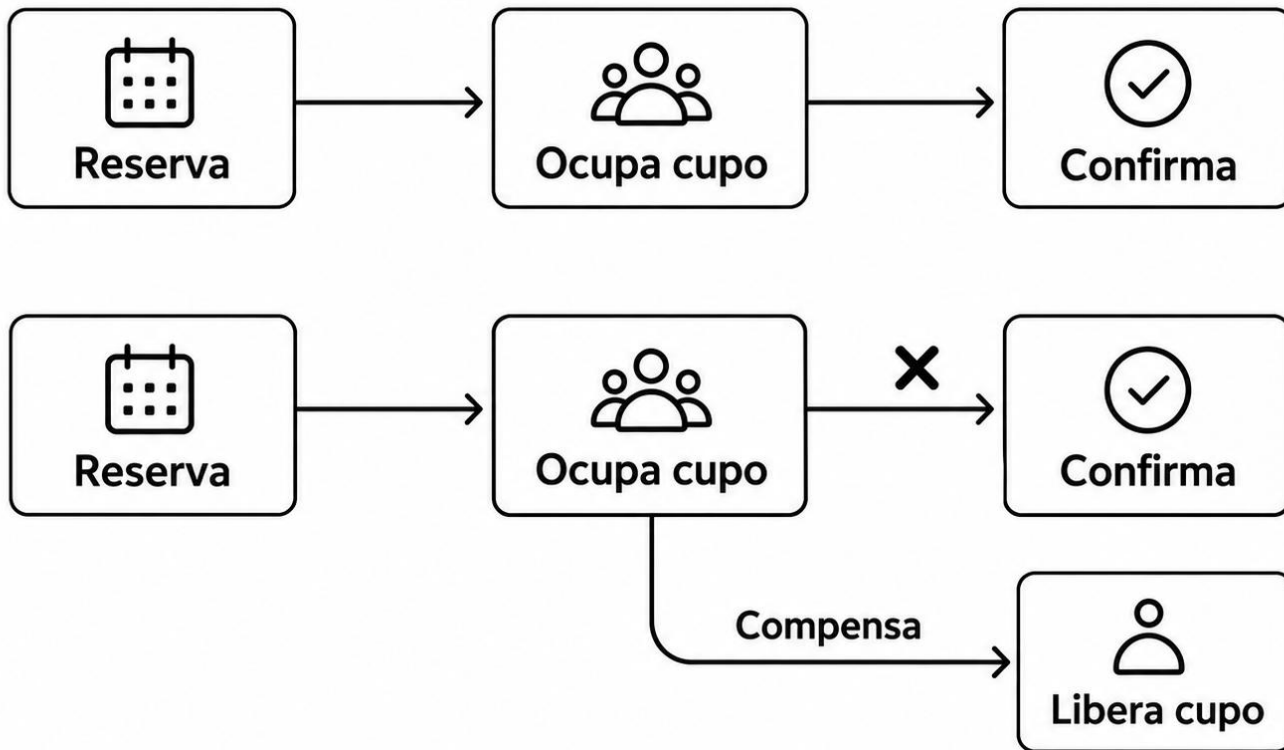
Airline



DECISIONES ARQUITECTÓNICAS

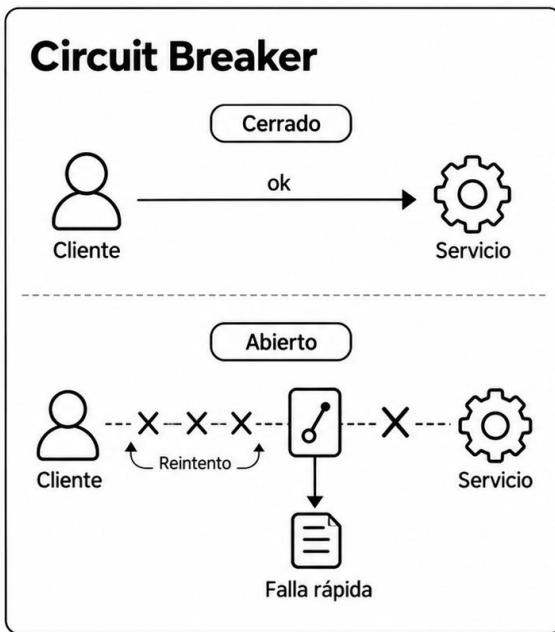
ADR	Decisión	Objetivo principal
ADR-001	Separar Vuelos, Pasajeros y Reservas	Modificabilidad
ADR-002	Usar gRPC + Protobuf	Desempeño y bajo acoplamiento
ADR-003	Cassandra con un keyspace por servicio	Separación de datos y escalabilidad
ADR-004	Astro como Frontend/BFF	Usabilidad y adaptación
ADR-005	Saga + Circuit Breaker	Fiabilidad y disponibilidad

SAGA Y COMPENSACIÓN



CIRCUIT BREAKER Y CUPO ATÓMICO

Circuit Breaker



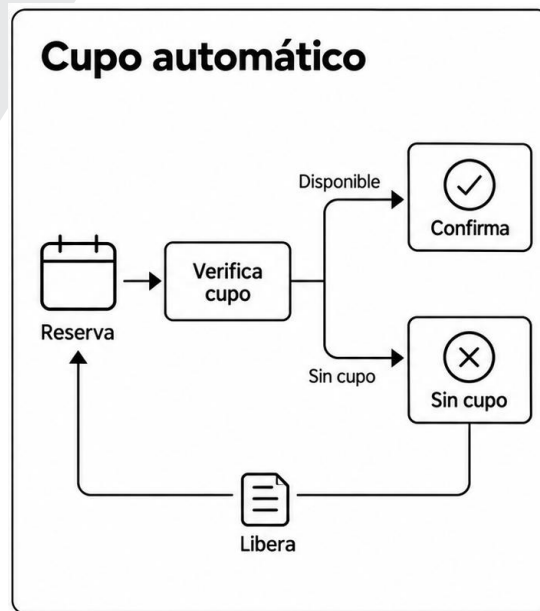
Circuit Breaker:

Booking → Flight
3 fallos → circuito abierto →
no llamar

Cupo:

Reserva → ocupa cupo →
confirma
si falla → libera cupo

Cupo automático





Airline



PRUEBAS Y EVIDENCIA

Saga

Compensación y fallo
ambiguo

Cupos

12 hilos, 3 cupos, 3 éxitos

Web

Reservas propias y
cancelación



Airline



EJECUCIÓN DEL STACK

docker compose up --build -d

docker compose ps

<http://127.0.0.1:4321>



Airline



DEMOSTRACIÓN EN VIVO

Catálogo

Identificación

Reserva

Mis reservas

Cancelar y comprobar cupo



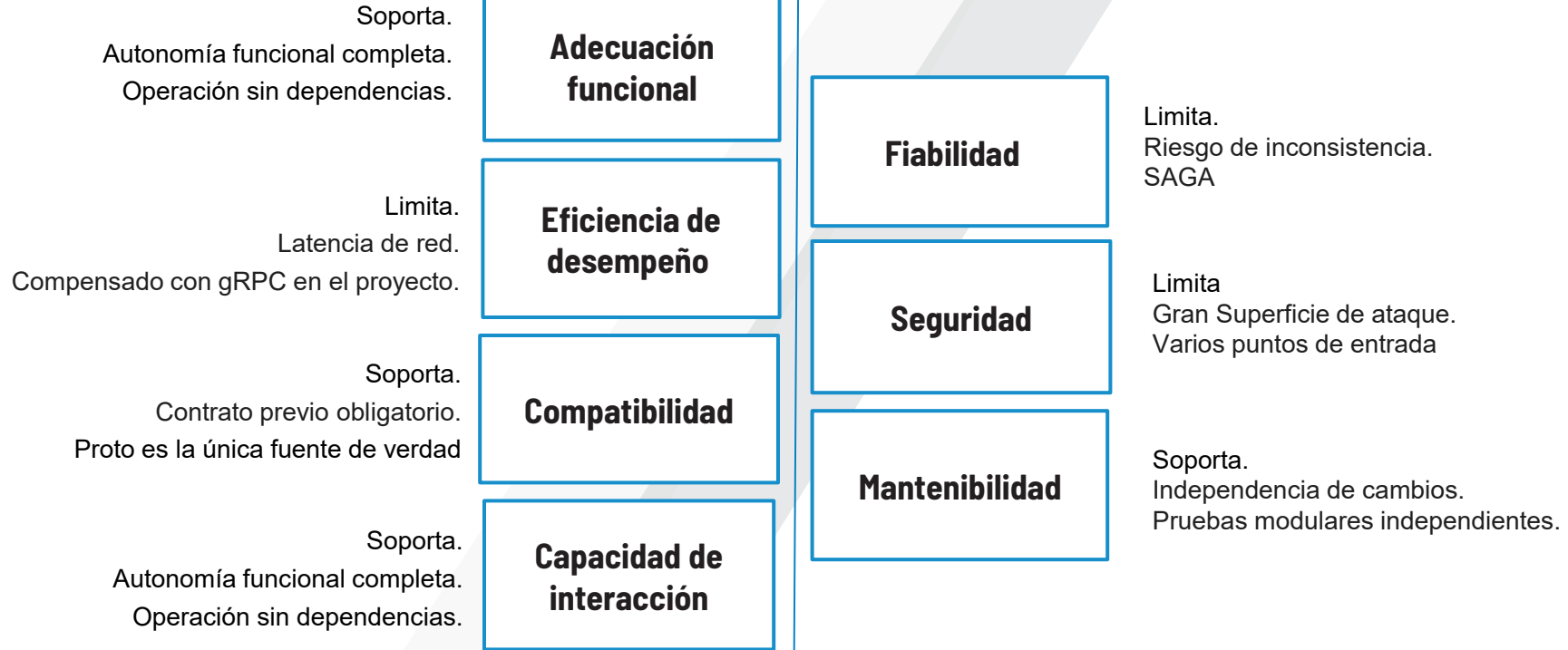
Airline



ANÁLISIS ARQUITECTÓNICO



Matriz de atributos de calidad vs Estilo



Matriz de tácticas vs estilo

ADR	Atributo	Táctica aplicada	En AeroReserva
ADR-001 Microservicios	Modificabilidad	Alta cohesión / bajo acoplamiento	Vuelos, Pasajeros y Reservas separados
ADR-002 gRPC + Protobuf	Desempeño / Modificabilidad	Contrato estable + menor overhead	.proto y comunicación binaria
ADR-003 Cassandra	Escalabilidad / Modificabilidad	Database per Service	Un keyspace por servicio
ADR-004 Astro BFF	Usabilidad / Modificabilidad	Intermediario BFF	Adapta Web ↔ gRPC
ADR-005 Resiliencia	Fiabilidad / Disponibilidad	Circuit Breaker + compensación	Corta fallos y libera cupos

Matriz de principios de diseño vs. estilo

Principio	En AeroReserva
SRP	Servicios separados
OCP	Resiliencia sin alterar lógica
Liskov	Contrato .proto estable
ISP	Métodos gRPC específicos
DIP	Dependencia del contrato
KISS	Docker Compose
DRY	Un solo .proto
YAGNI	Solo lo necesario
PoLA	Errores claros
Demeter	Sin acceso directo a datos
Composición	Circuit Breaker como envoltorio



Airline



GRACIAS

